Reasoning Graph (Local)

Deterministic, source-linked reduction of ReasoningFlow

Implementation whitepaper · Reasoning Workspace · 19 October 2026

Nathan Conklin
nathan.conklin@vt.edu
Department of Computer Science
Virginia Tech

Abstract

Reasoning Graph (Local) is a browser-computed view of an already annotated ReasoningFlow graph. It reduces visual complexity while retaining the membership of original nodes and links to the source text. The implementation uses two modes. When the full graph has at most 30 nodes, it preserves each node and relationship individually. Larger graphs are partitioned into consecutive groups, with the first and last generated reasoning nodes protected as singletons. Within each unprotected run, normalized TF-IDF vectors guide an initial threshold-based grouping pass and a subsequent adjacent-merge pass that enforces the allocated node budget. Labels are shortened extracts from representative nodes. Relationships are projected from the existing graph; internal relationships disappear from the compact view and equivalent external relationships are aggregated.

This paper explains the executed algorithm, its data contracts, pseudocode, runtime boundaries, layout, persistence, and limitations. It analyzes the user-selected OpenAI, fox, and Ricci-flow conversations by replaying their saved histories through the current application code. The Local computation makes no language-model request and invokes no FastAPI summarization endpoint. It is a source-linked discourse visualization, not a proof of reasoning correctness or a reconstruction of a model's internal causal computation.

Reading guide

Sections 1–3 explain the architecture and input. Sections 4–7 specify grouping and graph projection. Section 8 covers display and reuse. Section 9 presents the three real case studies. Section 10 discusses guarantees and limits. Appendices provide end-to-end pseudocode, reproducibility information, and source references.

Implementation baseline: graph algorithm 3; ReasoningFlow rules 4; embedded geometry 2; IndexedDB schema 4. The companion `algorithm.md` supplies the complete modular pseudocode and exact stop-word set.

1. Where the computation runs

The word “Local” describes the computation used to construct this graph, not the origin of every input. Chat responses and their raw reasoning can originate from a remote model through the FastAPI backend; research events can originate from backend-executed tools. Once that content is available, the Local graph pipeline performs annotation, grouping and layout in the browser. It does not send the trace back to a model. [1, 2]

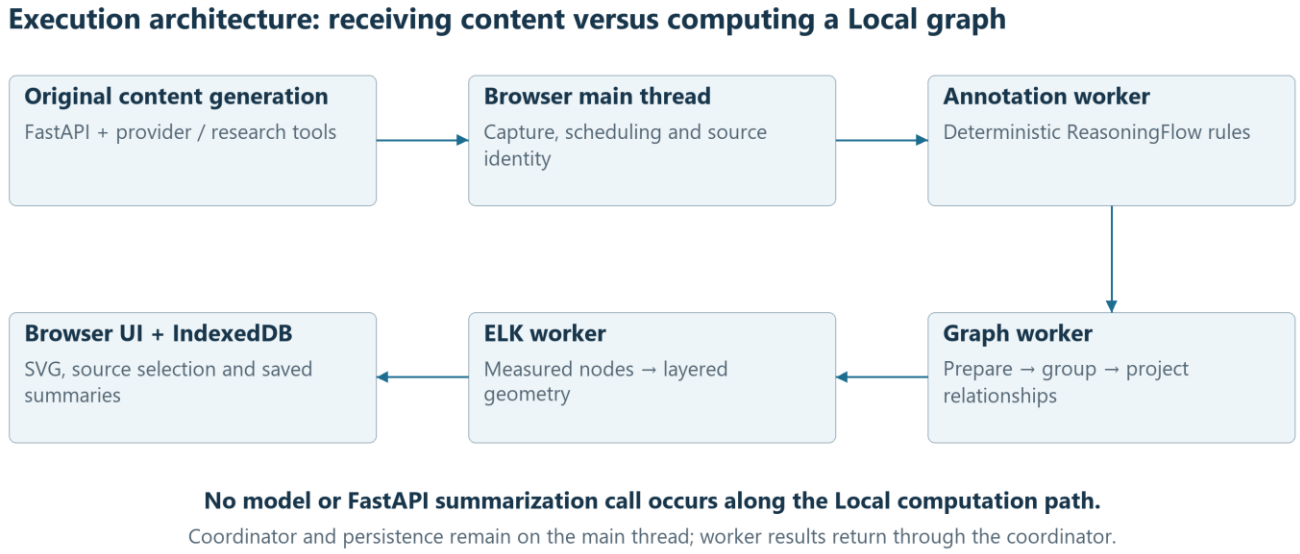


Figure 1. Original chat/tool generation is separate from local graph computation. Arrows show data flow, not additional provider requests. The Local path stops at browser rendering and storage.

Responsibility	Execution location	Model needed for this stage?
Receive chat reasoning and research events	Browser, supplied through FastAPI	Original chat may use a model
Capture and coordinate annotation	Browser main thread	No
Segment, classify and reconcile ReasoningFlow	Browser annotation worker	No
Prepare units, group and materialize Local graph	Browser graph worker	No
Measure labels and compute ELK geometry	Graph worker and ELK worker	No
Render SVG, source selections and tooltips	Browser main thread	No
Reuse and save derived summaries	Browser IndexedDB	No

The separate Reasoning Graph (Model) variant can call `/api/v1/reasoning-graphs/summarize`. That endpoint is not part of the Local algorithm. Even the Model variant bypasses provider summarization for graphs of 30 or fewer nodes. The Full ReasoningFlow Graph explorer is another local view: it displays the complete graph rather than applying the compact-card node budget. [2, 3]

2. The graph being simplified

ReasoningFlow is upstream of Local grouping. The original ReasoningFlow research describes a discourse graph schema and an annotation method; the project's tool extension adds operational and provenance records. The current application retains that vocabulary but replaces model annotation with deterministic client-side rules. Results reported for the research paper's annotation method are not measurements of these local rules. [4–6]

The annotation worker processes closed Markdown paragraphs or blocks and confirmed tool handoffs, then flushes the usable tail when reception ends. It preserves source IDs and original UTF-16 offsets. Explicit role rules run before contextual and fallback classification. The eleven node types are Context, Planning, Fact, Reasoning, Restatement, Assumption, Example, Reflection, Conclusion, ToolCall and ToolResult. A role such as Fact describes discourse use; it does not certify truth. [1, 5]

ToolCall and ToolResult nodes come from recorded activity, not from a sentence that merely says a search will occur. Returned excerpts can appear as Context. Available user/system invocation context may also be represented. Missing historical context manifests remain unknown; the replay pipeline does not invent them. Answer text in Thought is not used as a substitute for missing raw reasoning.

At terminal reconciliation, the annotation engine preserves precise/observed relationships and adds bounded contextual estimates: up to two contextual parents per generated node, plus marked fallback bridges between remaining reasoning components. Unused Context may receive an estimated uncertain association. These estimates carry rule evidence and are marked in the UI. [5, 7]

This distinction matters: upstream annotation may introduce an estimated relationship. Local graph grouping does not introduce a new relationship; it projects whichever relationships the upstream graph already contains. “No invented edges during grouping” therefore does not mean that every displayed edge is directly observed or logically verified.

Layer	What a node represents	What an edge represents
Full ReasoningFlow	A classified source unit or recorded context/tool record	A recorded, explicit, contextual or fallback relationship
Local full-detail mode	One original node	One original relationship
Local grouped mode	One consecutive interval of original nodes	One or more equivalent external original relationships

The current Local reducer neither contracts strongly connected components nor forces a single connected component. Those mechanisms appear in earlier ADRs but were superseded by actual-node membership and protected-interval grouping in ADR-041. [6, 8]

3. Preparing source-linked membership units

The `graphInput` function first sorts every actual ReasoningFlow node by recorded order, breaking ties with its stable ID. Each node becomes exactly one membership unit. A unit retains its original node ID, order, original source text, role weight, selectable ranges, and separate supplemental ranges. Context and tool nodes count toward the 30-node threshold even when they have no selectable passage. [2]

The unit's grouping text is the concatenation of original source slices, with the type name as a fallback. It is not the short label shown on screen. Local similarity can therefore use captured hidden Context when it is present. A separate `modelText` field is prepared from eligible selectable text for the Model variant; it is not the input to Local TF-IDF.

The reducer also builds an expected-coverage set from canonical, nonblank, selectable Reasoning Card sources. These include raw reasoning, research narration, and canonical tool fields. Tool payloads can be displayed both as serialized JSON and decoded field values. Alias mappings connect those representations, accounting for JSON escapes and UTF-16 boundaries rather than treating decoded positions as raw JSON positions.

If some eligible text is not covered by the original nodes' selectable ranges, it is attached as supplemental coverage. The first preference is the nearest original span from the same source. If none exists, recorded-order proximity selects an owner. Equal distances prefer the earlier unit. Only original spans participate in this lookup; a newly attached gap cannot attract another gap.

Supplemental text remains selectable but does not become classification evidence or add color weight. The reducer merges touching or overlapping ranges within each source; it does not modify stored text. If selectable text exists but no actual node exists, one neutral Unclassified coverage owner is created. If neither nodes nor eligible text exist, the coordinator reports that there is no content to summarize.

Two different notions of coverage

Membership coverage means every actual original node belongs to exactly one output group. **Selectable-text coverage** means the union of stored output ranges covers the expected selectable source ranges. These are independently checked. Neither implies that all text is printed in node labels, every passage has a correct role, or every semantic relationship has been captured.

Source ranges can overlap between different nodes. Membership is a partition of original node identities, not necessarily a partition of characters. A selected group can also point to multiple nonadjacent source ranges, even though its member nodes are consecutive in recorded node order.

4. Choosing detail and allocating the budget

Let `N` be the number of actual ReasoningFlow nodes and `K = 30`. For `N ≤ K`, each unit remains a singleton: no TF-IDF grouping is necessary, even if two units have identical text or are disconnected. Full-detail mode preserves original relationships, including parallel edges. Its labels are still shortened for display. [2, 8]

For `N > K`, the reducer finds the first and last generated nodes whose spans belong to reasoning or research text, excluding Context, ToolCall and ToolResult. These become protected singletons. They need not be classified as Planning or Conclusion; the last node is not promoted to Conclusion merely because it is last. Context or tool records can precede the first protected node or follow the last one.

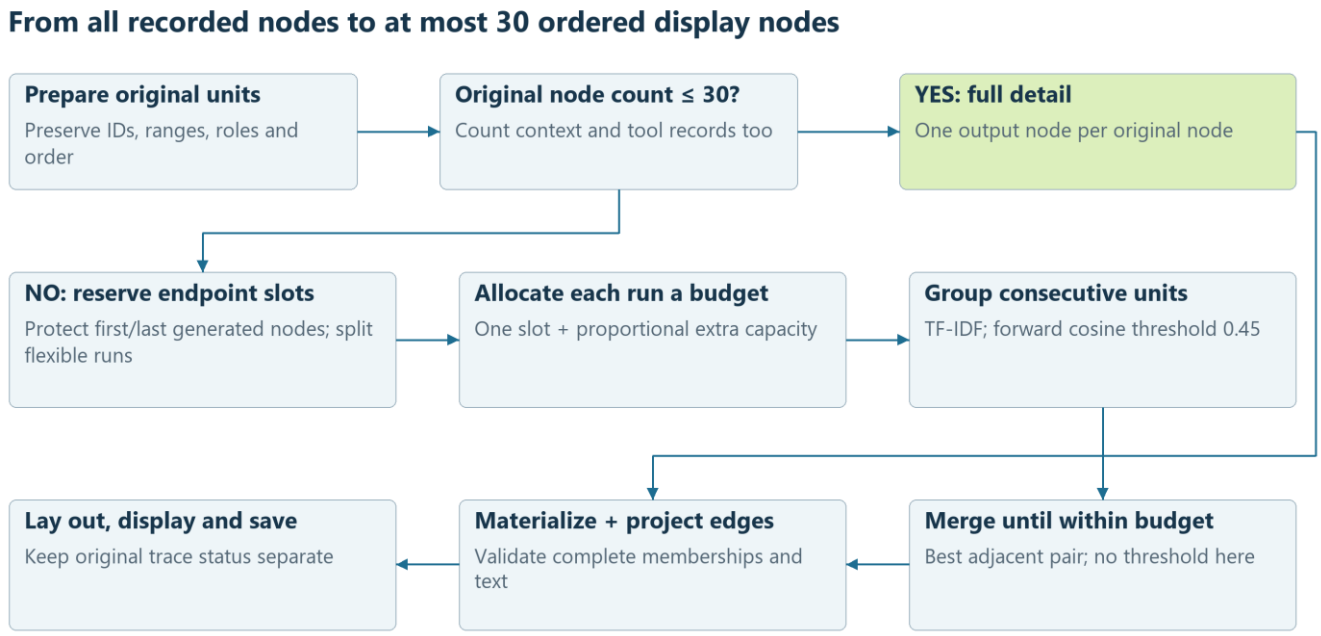


Figure 2. Protected endpoints act as barriers. No merge can cross them. The similarity threshold is used in the forward pass; the output budget is enforced separately.

Every maximal unprotected run initially receives one output slot. Protected runs also receive one slot. The remaining budget is distributed in proportion to each flexible run's extra capacity, `run length − 1`. Integer floors are assigned first; leftover slots go to the largest fractional remainders, with earlier runs winning ties.

The allocation is a maximum per run, not a promise to fill every slot. Initial threshold grouping can already fall below that maximum. The algorithm neither splits such groups to fill spare slots nor redistributes unused capacity across other runs. If a run already fits its allocated maximum, it is returned as individual nodes without calculating similarities.

5. Measuring lexical similarity

The implementation uses a sparse bag-of-words representation. TF-IDF makes terms that recur within one unit but occur in fewer units more influential. Cosine similarity compares normalized term distributions rather than raw text length. These are standard information-retrieval concepts; the exact smoothing, truncation and thresholds below are application choices. [9, 10]

Each run is its own document collection. The code lowercases unit text, extracts Unicode letter/number sequences at least three characters long, and removes a fixed English stop-word list. It does not stem words, expand synonyms, embed sentences, interpret equations, or call an LLM. Short identifiers, mathematical symbols and some negations can therefore disappear from this similarity representation even though the original source remains preserved.

For term `t` in unit `i`, let `tf(t,i)` be its occurrence count, `df(t)` the number of units in the run containing it, and `m` the number of units in that run. For present terms, the weight is:

```
w(t,i) = (1 + ln(tf(t,i))) * (1 + ln((m + 1)/(df(t) + 1)))
```

Absent terms have weight zero. Terms are sorted by decreasing weight, with lexical ties resolved by `localeCompare`; only the first 48 are retained per unit. That truncated vector is normalized to unit Euclidean length. An empty vector remains empty and has zero similarity to every vector.

A group centroid is the coordinate-wise sum of its normalized member vectors, normalized again. Although each member has at most 48 retained terms, a centroid may contain more than 48. Since vectors and centroids are normalized, their dot product is cosine similarity:

```
v[i] = TOP_48(w[:,i]) / (norm(TOP_48(w[:,i])) or 1)
c(G) = SUM(v[i] for i in G) / (norm(SUM(v[i] for i in G)) or 1)
similarity(a,b) = SUM(a[t] * b[t])
```

This favors lexical continuity, not logical equivalence. Two semantically opposed statements may share most of their words. Conversely, a synonym-based reformulation may score poorly. ToolCall, ToolResult and Context content participate through their original unit text; long returned payloads can influence a group's centroid and representative.

The document-frequency weights are calculated once for the run, not recomputed after each merge. Group centroids, however, are recalculated when comparisons are made.

6. Consecutive grouping and representative labels

The first pass scans a run in order. It compares each new unit only with the current last group. If similarity to that group's centroid is at least 0.45, the unit joins the group; otherwise it starts a new group. This is a greedy, order-constrained pass. An earlier group is never revisited to collect a recurring theme from a later passage. [2]

If too many groups remain, the second pass compares every adjacent pair of group centroids. The pair with the highest similarity is merged, and the scan repeats until the run meets its maximum. The comparison is strict: an exact tie leaves the earlier pair as the winner. The second pass has no minimum threshold, so a required merge can join zero-similarity groups.

```
LOCAL_RUN(units, maximum):
  if count(units) <= maximum: return singletons(units)
  vectors = TFIDF_WITHIN_THIS_RUN(units)
  groups = []
  for each unit i in recorded order:
    if groups exists and cosine(centroid(last group), vectors[i]) >= 0.45:
      append i to last group
    else: append new group [i]
  while count(groups) > maximum:
    j = earliest adjacent pair with greatest centroid cosine
    replace groups[j], groups[j+1] with their ordered union
  for each group:
    representative = earliest member with greatest cosine to centroid
    label = shorten(original text of representative)
  return groups with original memberships and labels
```

For a final group, the representative is the member whose vector is most similar to the centroid; exact ties select the earlier member. The label replaces a small set of Markdown-like punctuation with spaces, collapses whitespace, takes the first 12 words, and truncates to 110 JavaScript UTF-16 code units. Empty output falls back to “Recorded activity.” There is no generated paraphrase or semantic synthesis.

A label and a node color answer different questions. The label is selected by lexical centrality. The dominant color is selected by the sum of original role weights across every member; each original node contributes its source-span length, with a minimum of one. Ties use registry type order. A short representative can therefore carry a different role from the group's dominant role. Tooltips retain all represented roles, and supplemental text adds no role weight.

Thirty bounds only the displayed node count. The current implementation repeatedly constructs centroids; its grouping loops have a conservative quadratic cost in the run's node count, with the 48-term per-node cap bounding sparse-vector work. This is an algorithmic observation, not a measured performance guarantee.

7. Projecting relationships and preserving a DAG

Materialization gives each output node an A-number and retains its complete list of original member IDs, roles and source ranges. Validation requires that flattened memberships equal all input unit IDs exactly once and in order. Protected endpoints must remain singleton, and full-detail mode must contain only singletons. Coverage and role summaries are checked against the original units. Invalid input is rejected rather than silently repaired. [2]

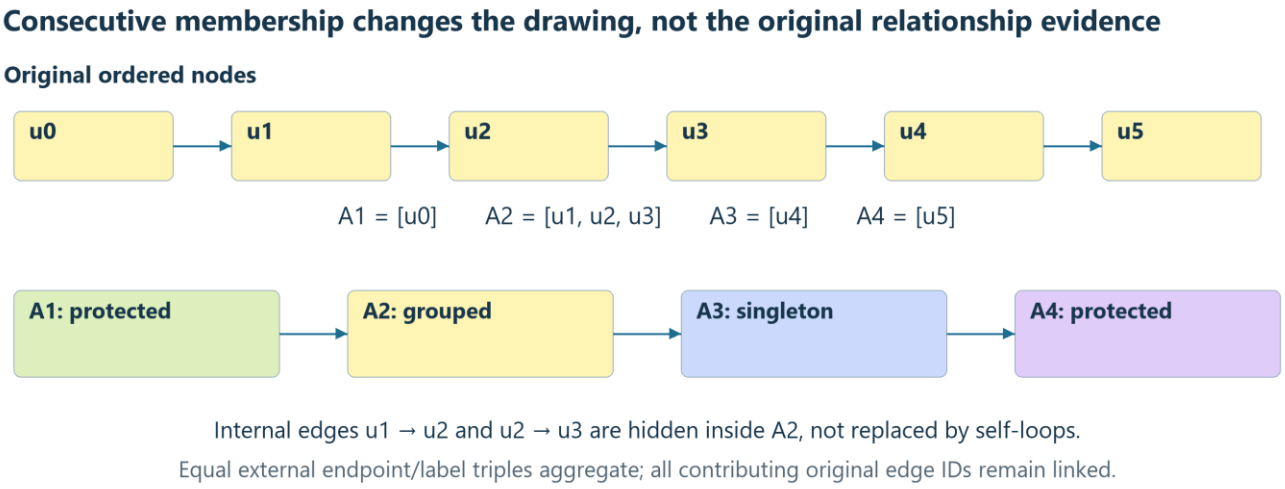


Figure 3. A schematic excerpt from a larger grouped graph, not a complete six-node input (which would retain full detail). Relationships inside a group disappear from the compact view. Equivalent external relationships aggregate; distinct labels remain distinct. The original graph remains available for inspection.

For every original edge, the reducer looks up the output owner of each endpoint. An edge whose endpoints share an owner is internal to that group and is omitted from the summary. Otherwise it is retained with its original label. In grouped mode, arrows with the same source group, destination group and label are aggregated, preserving all contributing original edge IDs. In full-detail mode, original edge IDs remain separate, preserving parallel edges.

No replacement “next” arrow is added merely because two groups are adjacent. No semantic relationship is synthesized from TF-IDF similarity. The full graph retains omitted internal relationships; the compact graph's external arrows do not enumerate them.

Why cycles cannot appear

The validator requires every original edge to move from a lower unit index to a higher one. Each group is a consecutive interval. For any surviving edge, its source interval must therefore precede its destination interval. Every surviving arrow moves forward through the ordered intervals, so a directed cycle is impossible. The proof depends on the validated order and consecutive membership; arbitrary nonadjacent topic clustering would not have the same guarantee.

Disconnected records are not deliberately discarded or force-connected. Grouping can nevertheless reduce the number of weakly connected components when one group contains original nodes from different components. That is shared ownership after reduction, not evidence of a new original relationship.

8. Display, persistence and lifecycle

The Local card starts computation only when its detail panel is selected and intersects the conversation viewport, reception has stopped, and ReasoningFlow reports complete. Hidden panels and the inert overview do not trigger jobs. Completed local processing and completed original response generation remain distinct: an incomplete or unknown historical trace can still be fully processed as available data. Canceled/deleted work is rejected. [2, 8]

The coordinator clones the available graph so background work does not depend on an evictable mutable cache. Its fingerprint includes graph/rule versions, variant, source identity, source text and ordering, original nodes and original relationships. A SHA-256 digest identifies the serialized input. Matching saved summaries are validated before reuse. Source or algorithm changes invalidate derived results; they do not rewrite original conversation text.

Geometry has its own version. Compatible memberships and labels with outdated geometry can be laid out again locally without regrouping. On completion, the coordinator first exposes the result and then attempts to save it to the conversation-indexed `reasoningGraphSummaries` store in IndexedDB version 4. A failed save leaves the graph visible with a warning. The save transaction checks the parent conversation, preventing a late result from recreating deleted conversation data. [2, 11]

Layout is a separate algorithm

Once memberships and arrows are fixed, ELK computes their coordinates. ELK Layered arranges graphs in layers and supports orthogonal routing; it is a layout engine, not a classifier or summarizer. [12, 13]

The application configures downward layout, orthogonal edge routes, Coffman–Graham layering with a layer-bound setting of three, network-simplex node placement, and random seed 1. Label measurement supplies node dimensions before layout. The renderer rounds route bends afterward; this changes drawing geometry, not graph relationships. [2]

The embedded card targets 128-pixel-wide nodes with 8-pixel type and measured wrapping. Width fitting can shrink that target further; height overflow scrolls vertically. Viewport changes adjust display scale without rerunning grouping. The full explorer uses a separate, larger layout and displays the complete graph, including disconnected records. These print figures enlarge and rearrange presentation for readability and are not screenshots of the browser UI. [11]

The production graph worker invokes ELK through an actual nested worker. Main-thread work still includes coordination, fingerprinting, IndexedDB access and SVG interaction; “runs in a worker” should not be read as “every operation is off the main thread.”

9. Case-study method and evidence

The user selected **OpenAI - Graph**, **Brother of a fox - Graph**, and **Ricci flow**, exported under the exact title **Ricci flow - Graph**. The export contains the selected conversations and four saved Local summaries. Only brief excerpts and derived graph information are reproduced here; full input and intermediate graphs remain in the ignored temporary directory.

Each conversation is reconstructed through the current `ReasoningFlowCoordinator.ensure` path, with the real annotation worker handler connected through the existing in-process test-port pattern. This preserves the application's historical replay ordering and terminal processing. The actual `graphInput`, `localGroups`, `materialize`, and membership validator then compute each Local graph. Network access is disabled in the analysis runner.

All eligible turns are inventoried. The selection rule is the completed turn with the greatest original node count, with earlier-turn tie-breaking. If no turn records completion, the largest eligible terminal trace is selected and its status is retained. All selected records here have **unknown original completion status**: the data can be completely processed without inventing a missing completion flag.

Case / selected turn	Original → Local nodes	Original → Local edges	Trace status
OpenAI - Graph / 1	44 → 30	61 → 49	unknown
Brother of a fox - Graph / 1	11 → 11	15 → 15	unknown
Ricci flow - Graph / 1	125 → 30	163 → 61	unknown

Each selected case is replayed twice and its semantic output compared. Edge accounting checks that every original edge either lies inside a group or contributes an original ID to an external projected edge. The analysis also compares saved summaries and records source fingerprint, membership, label and edge differences separately. A historical reconstruction is not assumed identical to a live graph merely because its node count matches.

The following figures show actual membership intervals, representative extracts and relationships from the selected turns. Any selected subgraph is identified as an excerpt; complete derived memberships and graph files accompany the analysis. The subject-matter statements inside excerpts are conversation data, not claims validated by this whitepaper.

9.1. OpenAI - Graph

Turn 1 · summary mode · original completion: unknown

The first generated node is protected at unit 0; the last generated node is protected at unit 40. Three tool-related records follow that endpoint. The 39-node middle run receives 26 slots, and the three-node tail receives two. This demonstrates why the protected ending is not necessarily the last displayed graph node.

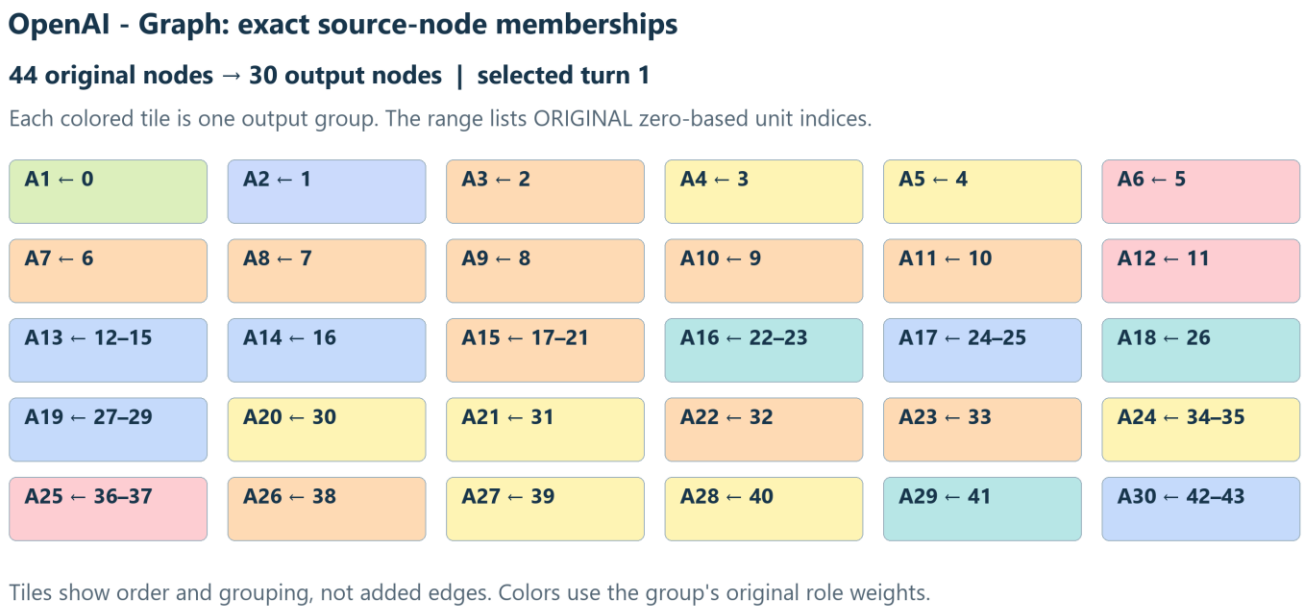


Figure 4. Reconstructed membership intervals; colors follow accumulated role weights. This is a membership visualization, not a set of adjacency edges.

Output	Original units	Dominant role	Exact representative label
A1	0	Restatement	The user asks about OpenAI API usage on the Pro plan ($200/month
A15	17–21	Fact	OpenAI announced in 2025 a "ChatGPT Pro" at $200/mo. There were reports
A28	40	Reasoning	Let me do one search for the help center page.

The reduction hides 10 internal relationships. The remaining 51 original edge references appear through 49 external arrows. Weakly connected components change from 2 to 1; shared group ownership can join components without creating a new original edge.

Saved source fingerprint, memberships and edges match the reconstructed output. The saved label for A16 differs from this replay; the cause has not been established. The paper uses replay labels consistently and does not claim identical cross-environment label output.

9.2. Brother of a fox - Graph

Turn 1 · full mode · original completion: unknown

The first turn contains 11 actual nodes, so all remain singleton. Its 15 original relationships remain 15 separate arrows; there are no internal edges to hide. The second turn contains five nodes and also uses full detail. No TF-IDF grouping is invoked for either turn. The ending is classified as Conclusion by the upstream rules, not by the reducer's endpoint position.

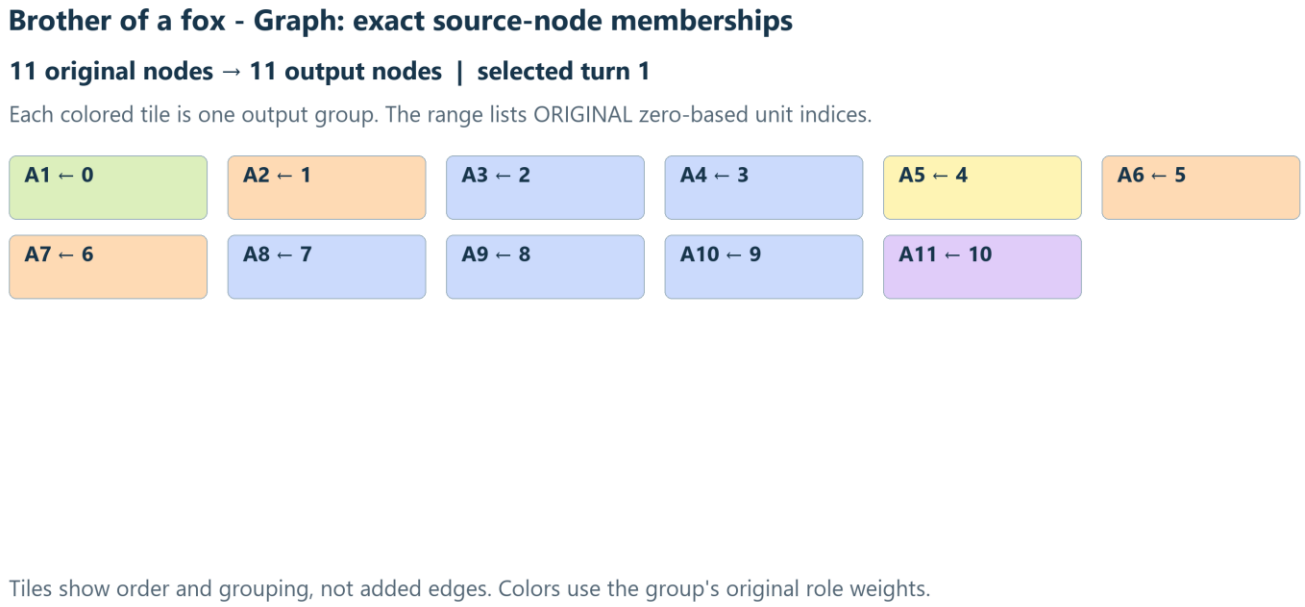


Figure 5. Reconstructed membership intervals; colors follow accumulated role weights. This is a membership visualization, not a set of adjacency edges.

Output	Original units	Dominant role	Exact representative label
A1	0	Restatement	"What is the brother animal of a fox?" — likely asking about
A11	10	Conclusion	I'll answer generally: in taxonomy, closest relatives.

The reduction hides 0 internal relationships. The remaining 15 original edge references appear through 15 external arrows. Weakly connected components change from 1 to 1; shared group ownership can join components without creating a new original edge.

Saved source fingerprint, memberships and edges match the reconstructed output. All saved representative labels match the replay.

9.3. Ricci flow - Graph

Turn 1 · summary mode · original completion: unknown

The first and last generated nodes are units 0 and 124. The 123-node interior receives 28 slots. Its groups vary greatly in size: the budget limits the number of display states, not the number of source units per state. The final protected node remains Reasoning rather than becoming a fabricated Conclusion.

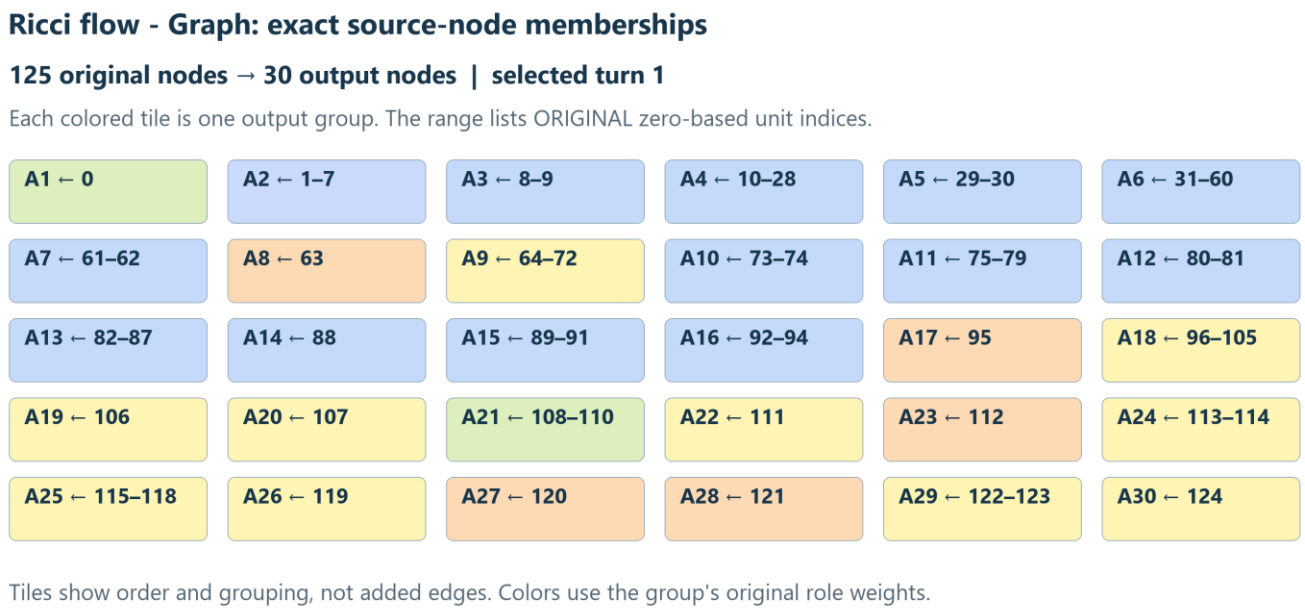


Figure 6. Reconstructed membership intervals; colors follow accumulated role weights. This is a membership visualization, not a set of adjacency edges.

Output	Original units	Dominant role	Exact representative label
A1	0	Restatement	The user asks about closed Ricci flow with bounded scalar curvature, and
A6	31–60	ToolResult	web search "openai/math" OR "Pitt" catalogue family 351 "Ricci-flow singularities" manuscript abstract
A30	124	Reasoning	Caveats and verification status

The reduction hides 76 internal relationships. The remaining 87 original edge references appear through 61 external arrows. Weakly connected components change from 11 to 1; shared group ownership can join components without creating a new original edge.

Saved source fingerprint, memberships and edges match the reconstructed output. The saved label for A12 differs from this replay; the cause has not been established. The paper uses replay labels consistently and does not claim identical cross-environment label output.

10. Guarantees, tradeoffs and observed limits

The strongest guarantees concern traceability and structure. Original node IDs remain represented; group memberships are ordered and exhaustive; protected endpoints survive; selectable text coverage is validated; and every displayed relationship traces to one or more original relationships. With valid forward edges and interval membership, the reduced graph remains acyclic. Source text is not rewritten by grouping. [2]

The compact view is not lossless as a drawing. Grouping hides internal node boundaries and internal edges. Short labels omit most member text. A dominant color can obscure minority roles. Distinct original branches can share a summary node, and connectedness can change under shared ownership. Inspecting the Full ReasoningFlow Graph or following source ranges is necessary to recover these details.

Lexical similarity does not measure entailment, conceptual identity or correctness. The fixed English stop-word list and minimum token length can remove distinctions important in mathematics or negation. Large tool payloads can affect representative selection. Endpoint protection can leave a weak or short final label; it intentionally protects the actual endpoint rather than inventing a more satisfactory conclusion.

Deterministic code still requires a specified execution environment for reproducibility. Some ties use locale-sensitive comparisons, and representative ranking uses floating-point cosine values without a tolerance. Repeated runs in the same analysis environment are checked; byte-identical labels across every JavaScript engine, locale or historical capture are not asserted. Saved-versus-replayed differences are disclosed in the case sections, not erased to make examples appear to match.

The three user-selected cases demonstrate behavior, not a representative semantic-accuracy study. No human-labeled benchmark, calibrated confidence score, factual evaluation or latency study is claimed. Automated tests establish specific contracts and regressions; they do not prove that inferred roles and relationships correctly explain a model's hidden computation.

Verification for this document build. Canonical lint passed. Canonical tests passed: 325 backend tests (one skipped), 647 web tests, 22 scrollbar-package tests, and ten browser-runner safety tests. The three selected cases each passed two independent historical replays, membership/coverage validation and original-edge accounting. No live model calls or automated host browsers were used. These counts describe contract checks, not semantic accuracy.

Earlier documentation must be read chronologically: ADR-038 described nonadjacent topic grouping and cycle contraction; ADR-039 introduced consecutive episodes and forced connectivity; ADR-041 replaced those reduction details with actual-node membership, protected endpoints and no forced connectivity. ADR-043 changed embedded geometry. This paper follows the latest approved behavior and current implementation rather than combining incompatible historical designs. [6, 8, 11]

Appendix A. End-to-end pseudocode

This listing connects the component algorithms. The complete operational reference, including source-gap ownership and exact tokenization, is supplied in `algorithm.md`.

```
BUILD_LOCAL_GRAPH(flow, sourceIdentity):
  REQUIRE reception stopped, annotation complete, visible detail card
  REQUIRE not canceled and parent conversation not deleted
  captured = clone(flow)
  fingerprint = HASH(algorithm version, rule version, local variant,
                     sourceIdentity, sources, nodes, edges, ordering)
  input = PREPARE(captured)
  REQUIRE input has at least one unit
  if saved summary passes fingerprint and membership validation:
    if geometry current: RETURN saved summary
    RETURN same semantic summary with refreshed local layout

  if original node count <= 30:
    groups = one singleton for each input unit
  else:
    protected = first and last generated reasoning/research units
    runs = maximal consecutive unprotected intervals,
           separated by protected singleton intervals
    budgets = ALLOCATE(runs, 30)
    groups = concatenate LOCAL_RUN(run, budget) in run order

  nodes = MATERIALIZE_AND_VALIDATE(input, groups)
  edges = PROJECT_ORIGINAL_EDGES(input.edges, nodes)
  layout = MEASURE_LABELS_AND_RUN_ELK(nodes, edges)
  summary = nodes + edges + layout + source identity + trace status
  DISPLAY summary
  SAVE summary only if parent conversation still exists
  if save fails: retain display and report save warning
  RETURN summary
```

Required invariants

- Every original membership index appears exactly once and in order.
- No group crosses a protected singleton; no full-mode node is grouped.
- Source ranges and role weights equal the values derived from members.
- Every expected selectable range is covered, independently of role confidence.
- Every input edge goes forward; every external output edge cites original edges.
- Output contains at most 30 nodes, except that empty input produces an error rather than a graph.

The zero-original-node/eligible-text case is explicit: a neutral owner preserves coverage without inventing a classified ReasoningFlow node. It is the only reason full mode can display one node when the original node count is zero.

Appendix B. Budget and edge algorithms

```
ALLOCATE(runs, K):
  maximum[r] = 1 for every run, including protected singletons
  R = K - count(runs)
  capacity[r] = length(r) - 1 for flexible runs only
  C = SUM(capacity[r])
  quota[r] = R * capacity[r] / C if C > 0 else 0
  maximum[r] += FLOOR(quota[r])
  left = R - SUM(FLOOR(quota[r]))
  visit flexible runs by fractional quota descending, earlier ties:
    if left > 0 and maximum[r] < length(r):
      maximum[r] += 1; left -= 1
  RETURN maximum

PROJECT_ORIGINAL_EDGES(edges, nodes):
  owner = map each original unit to its containing output node
  output = ordered map
  for edge in edges:
    source = owner[edge.source]
    destination = owner[edge.destination]
    if source == destination: CONTINUE
    key = original edge ID if full mode
          else (source, destination, edge.label)
    if key already exists:
      add original edge ID to output[key].originals if absent
    else:
      output[key] = (source, destination, edge.label,
                     originals = [original edge ID])
  RETURN output values
```

The capacity formula handles runs before, between and after generated endpoints. No second allocation pass redistributes slots after grouping. An exact tie during an adjacent merge favors the earlier pair; an exact representative tie favors the earlier member. Group colors instead break equal accumulated role weights by registry order.

Reproducing the cases

Run `node scripts/analyze-reasoning-graph-local.mjs` with the selected export at `temp/reasoning-graph-local-cases.json`, or supply its path as the first argument. The runner loads production TypeScript through the installed Vite loader, uses the application worker handler in process, disables fetch, and writes private derived evidence under `temp/reasoning-graph-local/`.

This analysis environment executes the browser's computation modules without launching a browser or calling a provider. It tests the algorithm and historical replay; it is not a browser rendering benchmark. The Word build uses the supplied Markdown and local figure sources. Its PDF preview is an inspection artifact, not a replacement for the editable Word document.

References and implementation map

[1] Project architecture, G44–G53; `src/web/src/reasoningFlow/coordinator.ts`, `annotation.ts`, `boundaries.worker.ts`, `capture.ts`, `relationships.ts`. Current browser annotation and historical replay.

[2] `src/web/src/reasoningGraph/grouping.ts`, `coordinator.ts`, `graph.worker.ts`, `types.ts`, `payloadRanges.ts`, `Card.tsx`, `layout.ts`, `nodeText.ts`. Executed Local input, grouping, projection, lifecycle and drawing contracts.

[3] `src/api/routes/reasoning_graph.py` and `src/web/src/reasoningGraph/model.ts`. Separate Model summarization path and its local bypass.

[4] Lee, J., Agarwal, S., Parulekar, A., Madala, S., Hakkani-Tür, D., and Hockenmaier, J. (2026). *ReasoningFlow: Discourse Structures for Understanding LLM Reasoning Traces*. arXiv:2606.05402v1. Local supplied PDF, especially Sections 3–4 and Appendix C.1. Referenced for schema/method background, not as an evaluation of this application's rules.

[5] `docs/reasoning-flow-deterministic-rules.md` and ADR-034 through ADR-036; ADR-040 and its version-four addendum supersede earlier sparse-only relationship restrictions.

[6] Conklin, N. (2026). *ReasoningFlow with Tool Calls and Web Search: Incremental Annotation of Streaming Reasoning Content*. Local supplied design PDF. ADR-038 and ADR-039 record earlier graph-summary designs.

[7] ADR-040, *Selective estimated ReasoningFlow relationships*. Accepted 13 October 2026.

[8] ADR-041, *Faithful Reasoning Graph detail and inspection*. Accepted 13 October 2026. Read together with G51 requirements and specification.

[9] Manning, C. D., Raghavan, P., and Schütze, H. (2008). *Introduction to Information Retrieval*, “Tf-idf weighting.” Author-hosted text. Background only; the application-specific formula is specified in Section 5.

[10] Manning, Raghavan, and Schütze (2008), “Dot products.” Author-hosted text. Normalization and cosine background.

[11] ADR-042, *Compact readable Reasoning Graph cards*; ADR-043, *Vertical-only compact Reasoning Graphs*; `src/web/src/storage.ts`. Presentation and persistence.

[12] Eclipse Layout Kernel. ELK Layered reference. Official layout-algorithm documentation.

[13] Kiel University / ELK contributors. elkjs repository. JavaScript layout engine; the project pins elkjs 0.11.0. Online references accessed 19 October 2026.

Repository baseline: `0e7c78ff83fbe909e20c06fbaadaa1f3ab2cda0a`. The working tree contains pre-existing changes; file hashes identify the reviewed bytes.

Implementation file	SHA-256 prefix (16 hexadecimal digits)
grouping.ts	`f0f2d80d0ccd159f`
coordinator.ts	`fb6c086066d80a0a`
graph.worker.ts	`8888b02d3f1bc318`
types.ts	`a0f44e0dcc59c480`
payloadRanges.ts	`2affabd20e58d197`
layout.ts	`8c4db67f8209b2fa`

The complete file-hash manifest is in `output/reports/reasoning-graph-local/source-manifest.json`. Conversation export and private reconstructed graphs remain under `temp/`; they are not repository fixtures.